CPU Scheduling Algorithms

Priority Scheduling vs. Round Robin (RR)

Threads and SMP

Multithreading & Symmetric Multiprocessing

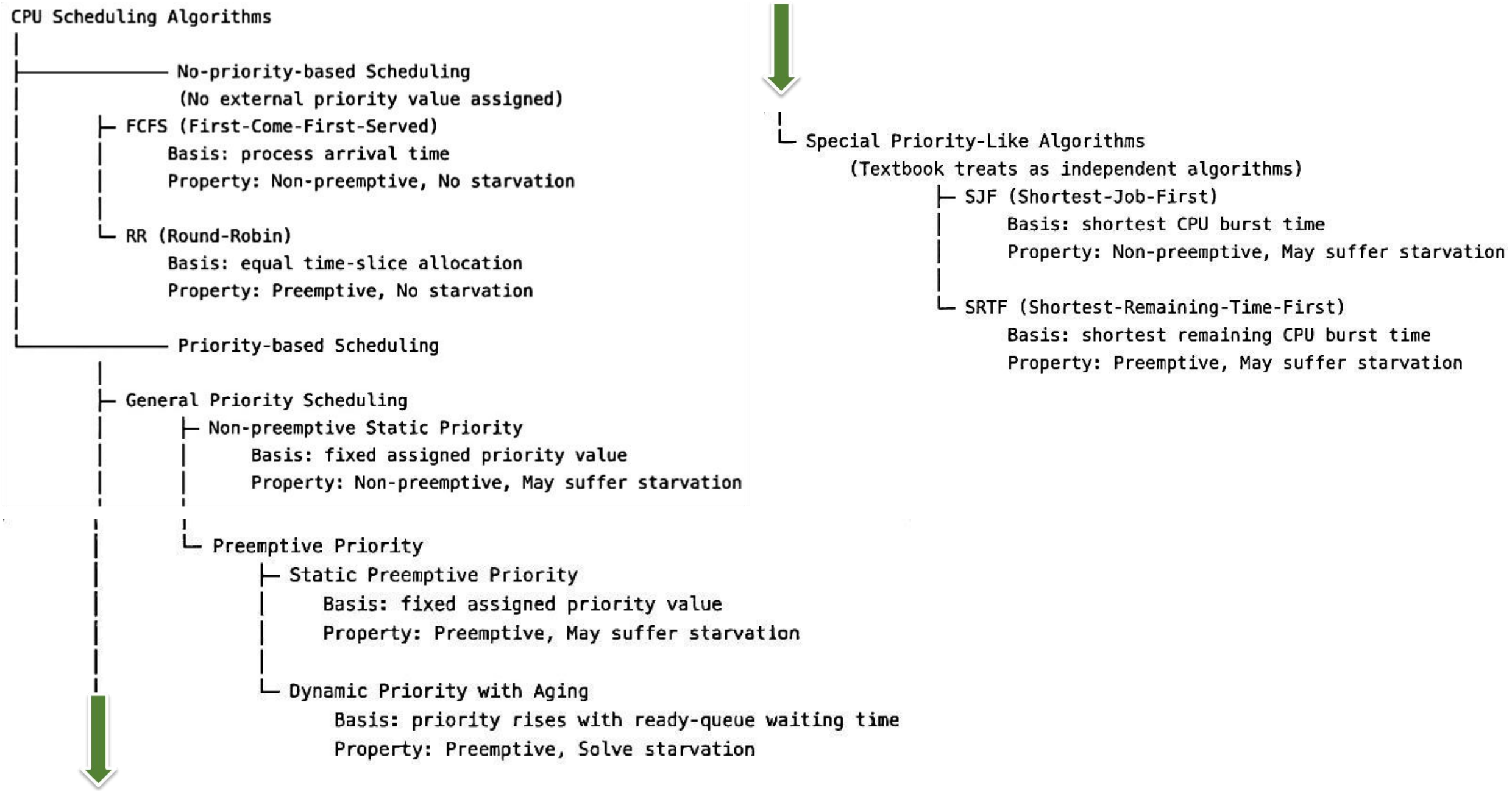
COMP2113 · OPERATING SYSTEMS

CPU Scheduling Algorithms

Priority Scheduling vs. Round Robin (RR)

CPU scheduling is the basis of multiprogrammed operating systems. This section compares two fundamental scheduling algorithms and their trade-offs.

CPU Scheduling Algorithms Hierarchy



Round-Robin (RR) Scheduling

Designed specifically for **time-sharing systems**. Processes are placed in a **circular ready queue**.

How It Works

- A small time unit called the **time quantum** is defined (typically 10–100 ms).
- The scheduler selects the first process from the queue and executes it for **one time quantum**.
- If **burst time < quantum**: process finishes, CPU moves to the next process.
- If **burst time > quantum**: process is interrupted via **context switch** and placed back at the **tail** of the queue.

Time Quantum Trade-off

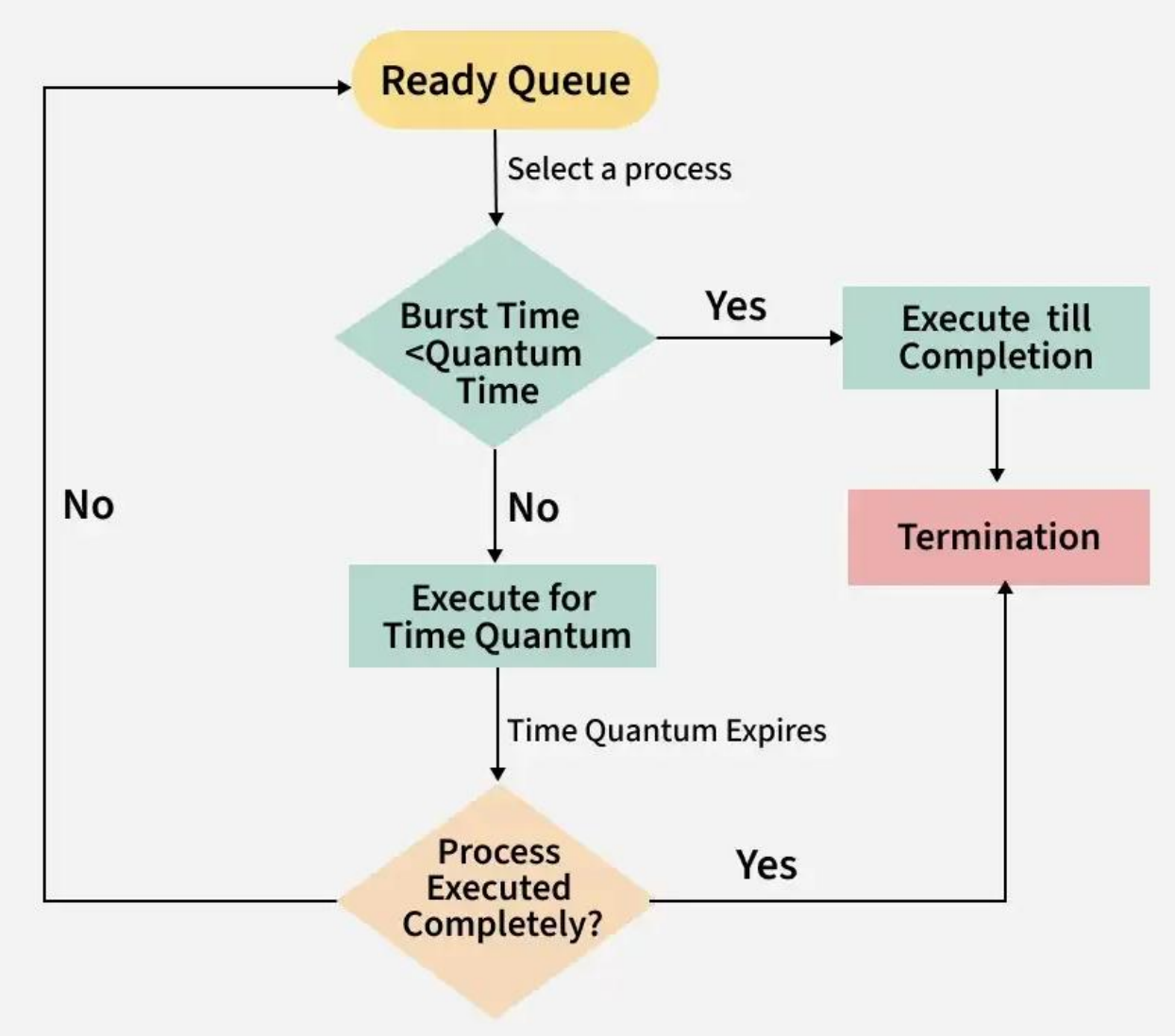
Very large quantum → RR behaves like **FCFS**.
Very small quantum → frequent context switches, high overhead.

Key Characteristics

Preemptive in nature.
No starvation — all processes get CPU time in turn.

Disadvantage: Quantum size must be carefully chosen — larger quantum increases waiting times for short processes; smaller quantum increases context-switch overhead.

Round-Robin (RR) Flow Chart



Round Robin Scheduling is a method used by operating systems to manage the execution time of multiple processes that are competing for CPU attention. It is called "round robin" because the system rotates through all the processes, allocating each of them a fixed time slice or "quantum", regardless of their priority.

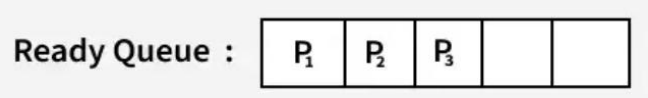
The primary goal of this scheduling method is to ensure that all processes are given an equal opportunity to execute, promoting fairness among tasks.

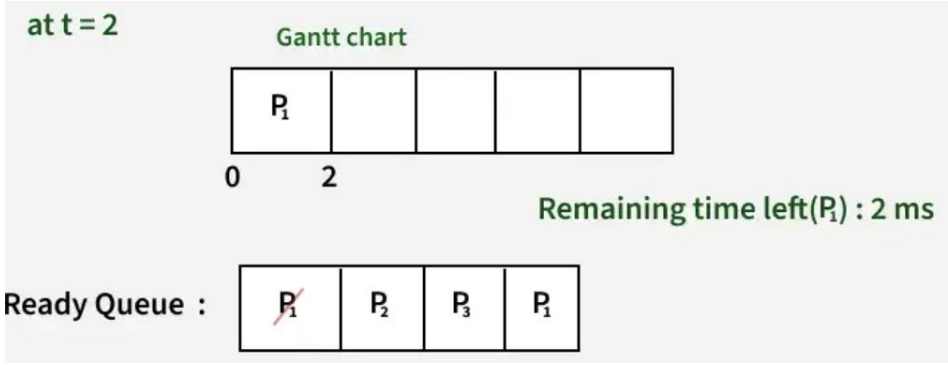
1. **Process Arrival:** Processes enter the system and are placed in a queue.
2. **Time Allocation**: Each process is given a certain amount of CPU time, called a quantum.
3. **Execution:** The process uses the CPU for the allocated time.
4. **Rotation:** If the process completes within the time, it leaves the system. If not, it goes back to the end of the queue.
5. **Repeat:** The CPU continues to cycle through the queue until all processes are completed.

Scenario 1: Processes with Same Arrival Time

Consider the following table of arrival time and burst time for three processes P1, P2 and P3 and given Time Quantum = 2 ms

Process	Burst Time	Arrival Time
P1	4 ms	0 ms
P2	5 ms	0 ms
P3	3 ms	0 ms

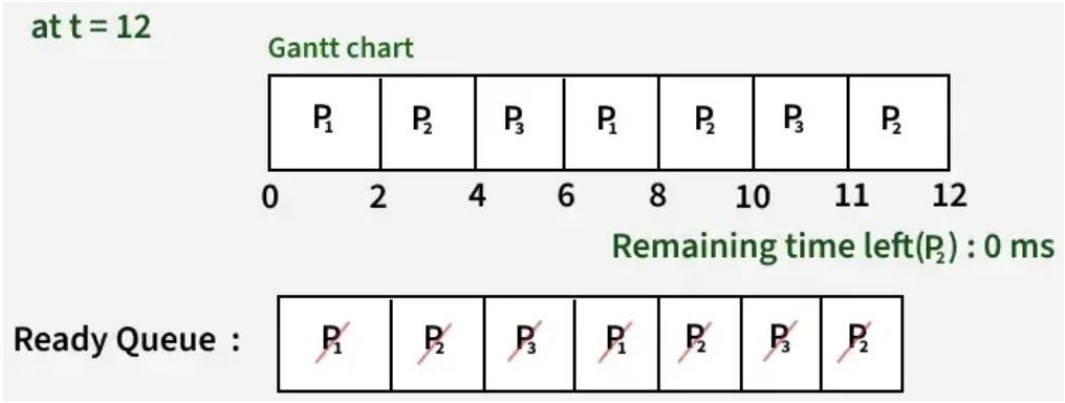




∘ ∘ ∘

Step-by-Step Execution:

Time 0-2 (P1): P1 runs for 2 ms (total time left: 2 ms).
Time 2-4 (P2): P2 runs for 2 ms (total time left: 3 ms).
Time 4-6 (P3): P3 runs for 2 ms (total time left: 1 ms).
Time 6-8 (P1): P1 finishes its last 2 ms.
Time 8-10 (P2): P2 runs for another 2 ms (total time left: 1 ms).
Time 10-11 (P3): P3 finishes its last 1 ms.
Time 11-12 (P2): P2 finishes its last 1 ms.



Priority Scheduling Algorithm

Executes processes according to their **priority**. Each process is allocated a priority; the process with the **highest priority** is executed first.

Priority Types

Internal: decided by the system — number of resources required, time needed, etc.
External: based on importance of work, amount paid, or deadline urgency.

Preemptive Modes

Preemptive: higher-priority process can preempt a running one.
Non-preemptive: running process keeps CPU until it finishes or blocks.

Note: If two processes have the same priority, the tie is broken using **FCFS**. In preemptive mode, the waiting time for the highest-priority process is always **zero**; in non-preemptive mode it may not be zero.

Disadvantage — Starvation: Low-priority processes may never get executed if the system keeps handling a stream of high-priority processes (Under priority-based scheduling, low-priority processes residing in the ready queue may suffer starvation). Thinking: solution?

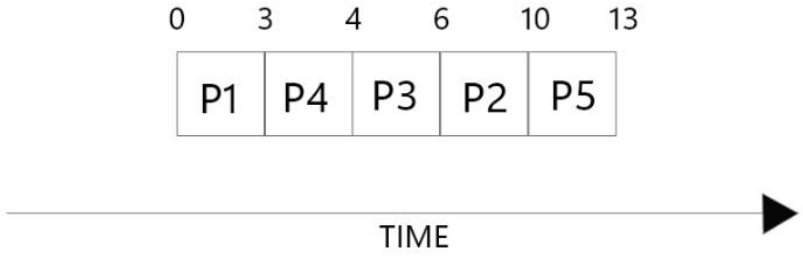
Priority Scheduling vs. Round Robin

Criteria	Priority Scheduling	Round Robin (RR)
Core Principle	Highest-priority process runs first	Each process runs for a fixed time quantum in turn
Preemption	Can be preemptive or non-preemptive	Always preemptive
Starvation	Yes — low-priority processes may starve	No — all processes get CPU time in turn
Key Parameter	Priority assignment (internal / external)	Time quantum size
Best For	Systems where task importance varies (real-time, batch)	Time-sharing / interactive systems
Main Drawback	Starvation of low-priority processes	Quantum too large → FCFS; too small → high overhead

Takeaway: Priority scheduling optimizes for importance; RR optimizes for fairness. Modern OS schedulers often combine both (e.g., priority-based RR within each priority level).

Priority Scheduling (Non-Preemptive)

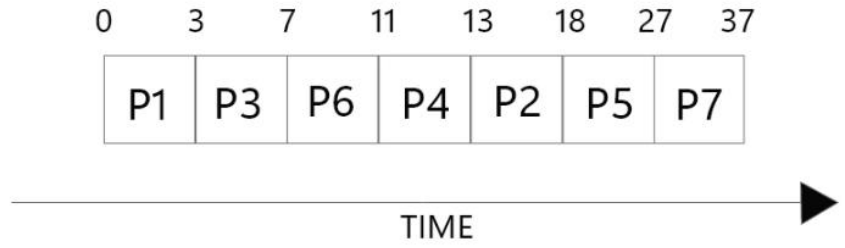
Priority Scheduling Algorithm



Process	P1	P2	P3	P4	P5
Burst Time	3	4	2	1	3
Priority	1	3	2	1	4

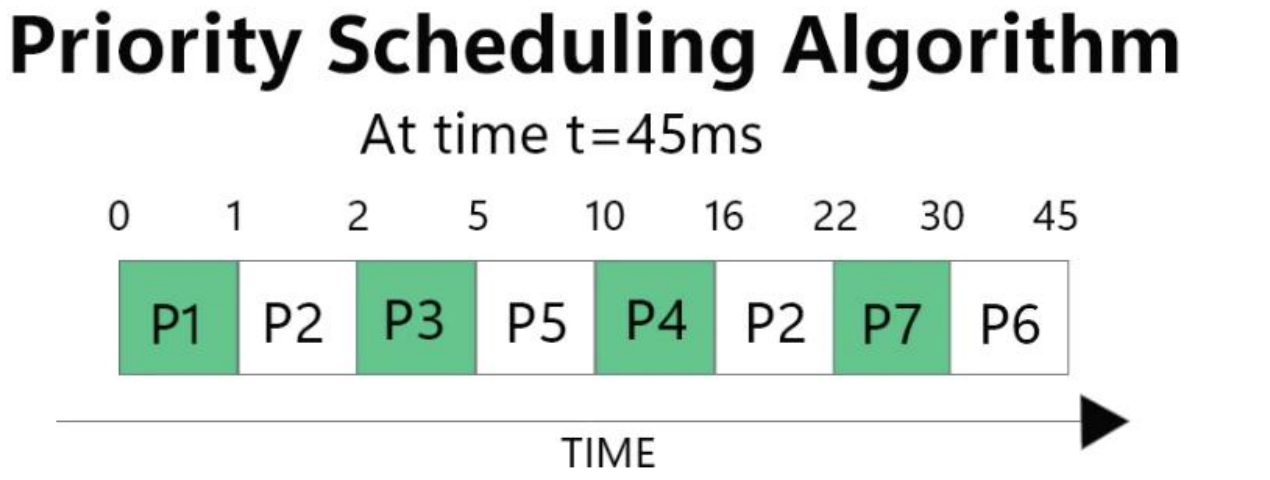
1. Both P1 and P4 have the highest Priority(1). Thus we will use FCFS to settle clash and P1 will execute first.
2. P4 will execute once P1 is finished executing
3. P3 has priority 2 thus it will execute next.
4. Then finally P2 and P5

Priority Scheduling Algorithm



Process	P1	P2	P3	P4	P5	P6	P7
Burst Time	3	5	4	2	9	4	10
Priority	3	6	3	5	7	4	10
Arrival Time	0	2	1	4	6	5	7

Priority Scheduling Algorithm （Preemptive）



Process	P1	P2	P3	P4	P5	P6	P7
Burst Time	1	7	3	6	5	15	8
Priority	2	6	3	5	4	10	9
Arrival Time	0	1	2	3	4	5	15

1. Turnaround Time = Completion Time – Arrival Time
2. Waiting Time = Turn Around Time – Burst Time

COMP213 · OPERATING SYSTEMS

Threads and SMP

Chapter 4 — Multithreading & Symmetric Multiprocessing

Why Do We Need Threads?

- **Responsiveness** — It allows applications to be responsive to user input i.e., other threads can react to user's action if one thread is busy.
- **Resource Sharing** — Due to memory and resource sharing, there is efficient communication without inter-process communication.
- **Cost Efficient** — To create and manage threads, it requires less resources and time compared to creating separate processes.
- **Parallelism** — Threads implement parallelism on multi-core and multiprocessor systems. It improves the performance.
- **Better CPU Utilization** — When one thread is blocked for I/O operations, other threads can continue execution. It helps in maximizing CPU usage.

Components of a Thread

The key components of a thread are mentioned below —

- **Thread id** — It represents a unique identifier for each thread to differentiate one thread from other. The operating system uses this id to track and manage the threads.
- **Program Counter** — It is a register that holds the address of the next instruction that will be executed by the thread. Each thread has its own program counter.
- **Register Set** — It is a collection of registers that holds the current working variables and temporary data for the thread. Each thread maintains its own set of register values.
- **Stack** — It stores local variables, function parameters, and return addresses for the thread. Each thread has its own stack. The stack of the thread maintains execution history, function call sequence, and prevents interference with other threads.

Types of Threads

An operating System is divided into two spaces: User Space and Kernel Space.

User-Level Threads(ULTs)

ULTs or User-level threads are implemented in user space by thread libraries. In this, the kernel is not aware of ULTs. Kernel manages them just like a single threaded processes. In this space, operations like thread creation, scheduling, synchronization is done.

Kernel-Level Threads(KLTs)

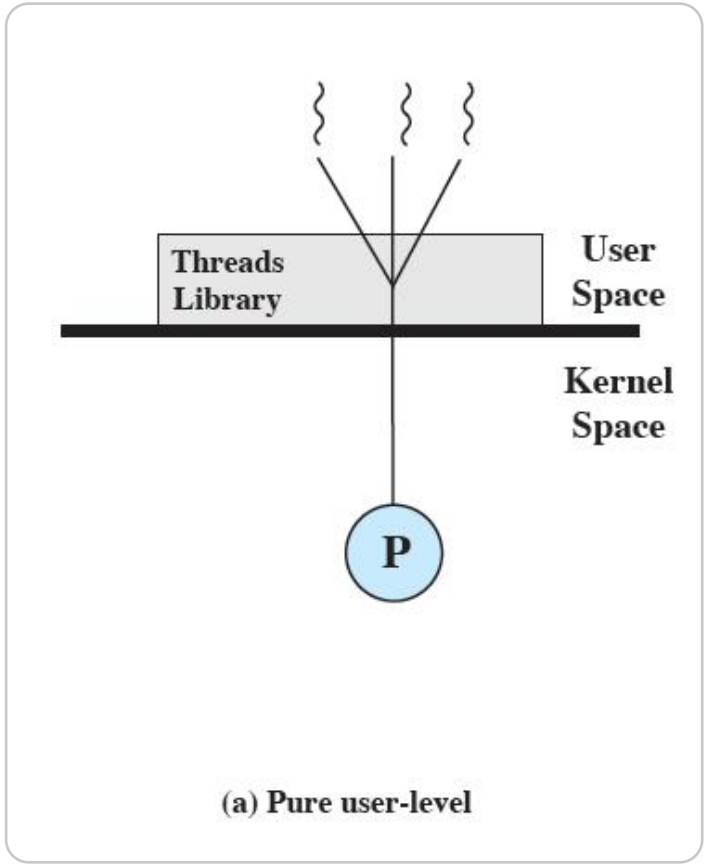
The kernel of operation system directly manages KLTs or Kernel-level threads. The kernel maintains thread information and also performs operations like thread creation, scheduling, and management. In this space, the kernel is aware of all threads in the system unlike ULTs.

Difference between User-Level & Kernel-Level Thread

S.N.	User-Level Threads	Kernel-Level Thread
1	User-level threads are faster to create and manage.	Kernel-level threads are slower to create and manage.
2	Implementation is by a thread library at the user level.	Operating system supports creation of Kernel threads.
3	User-level thread is generic and can run on any operating system.	Kernel-level thread is specific to the operating system.
4	Multi-threaded applications cannot take advantage of multiprocessing.	Kernel routines themselves can be multithreaded.

User-Level Threads (ULTs)

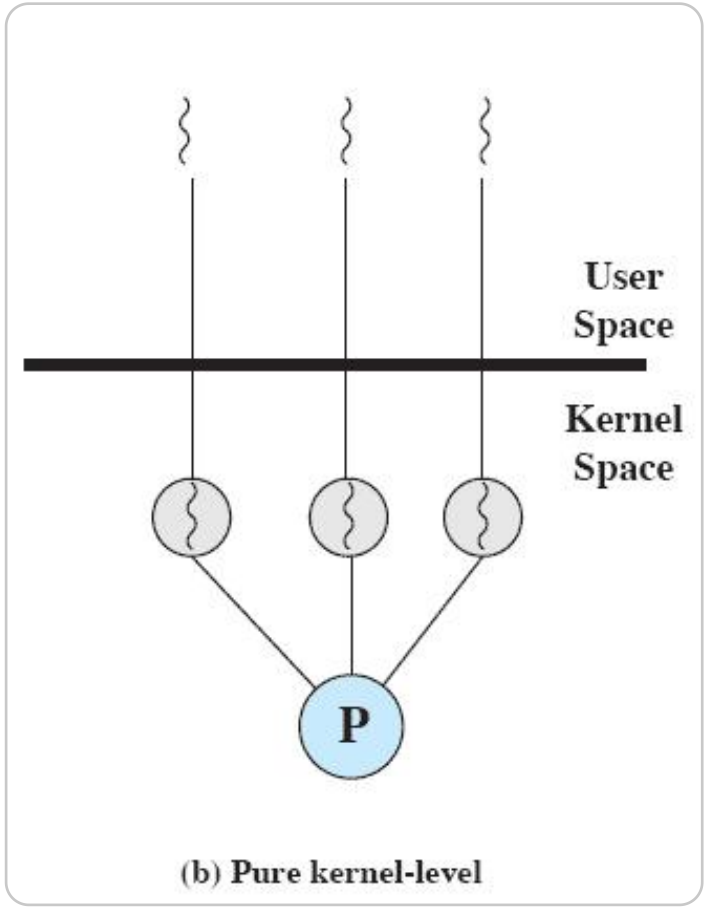
- One of the **two types** of thread implementations
- All thread management is done by the **application**
- The kernel is **not aware** of the existence of threads
- Thread switching happens in user space — no kernel mode transition
- Advantage: fast switching, portable across OS
- Disadvantage: one blocking call blocks the entire process



ULT Architecture Diagram

Kernel-Level Threads (KLTs)

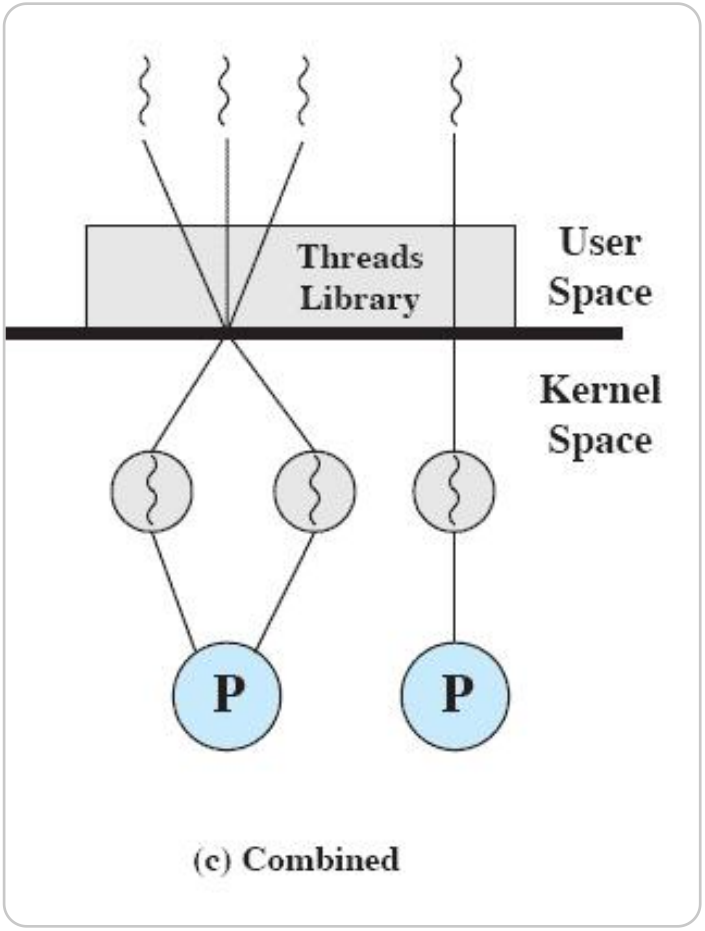
- Thread management is done by the **kernel**
- **No** thread management is done by the application
- The kernel maintains context information for both processes and threads
- Thread switching requires **kernel mode transition**
- Advantage: one thread blocking does not block the whole process
- *Example: Windows*



KLT Architecture Diagram

Combined Approaches

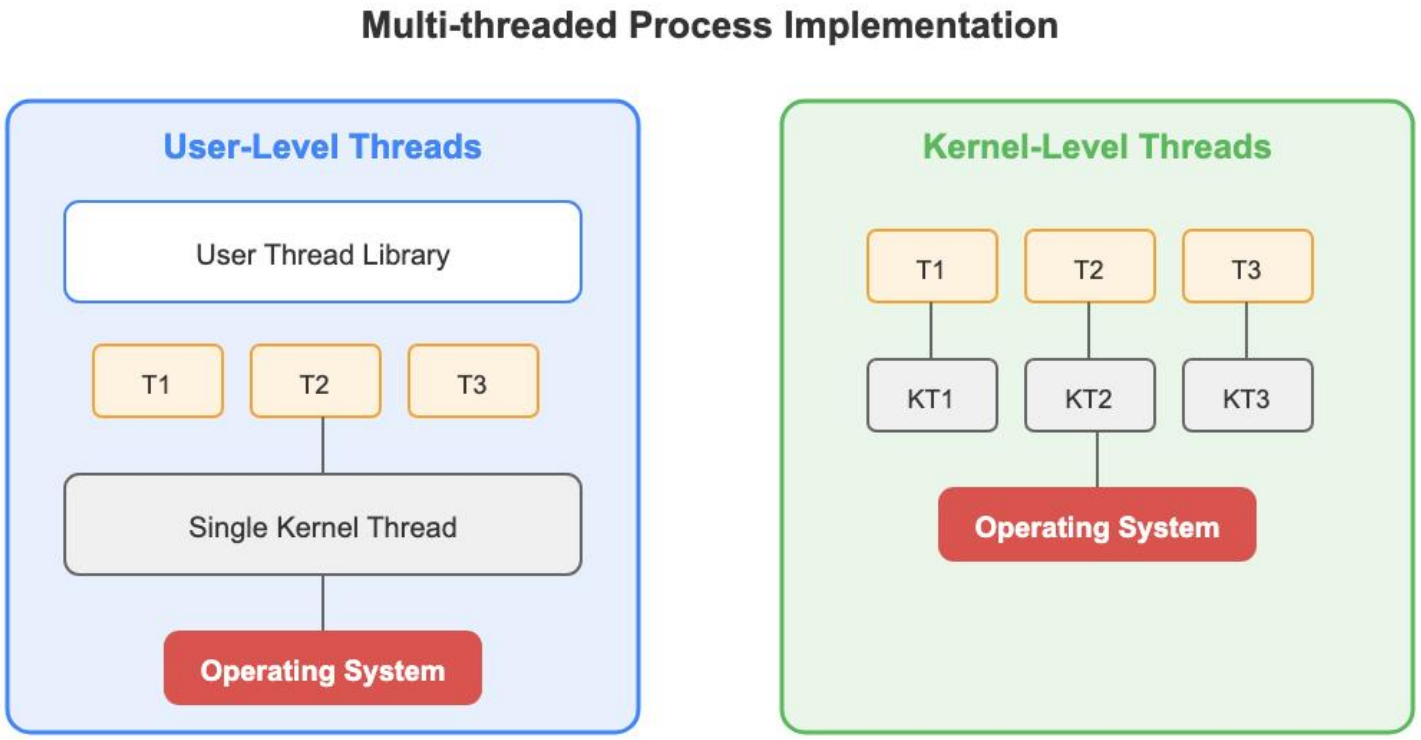
- Thread creation is done in **user space**
- Bulk of scheduling and synchronization is done by the **application**
- Maps user-level threads to kernel-level threads (many-to-many model)
- Combines advantages of both ULT and KLT
- Application manages thread creation; kernel handles scheduling
- *Example: Solaris*



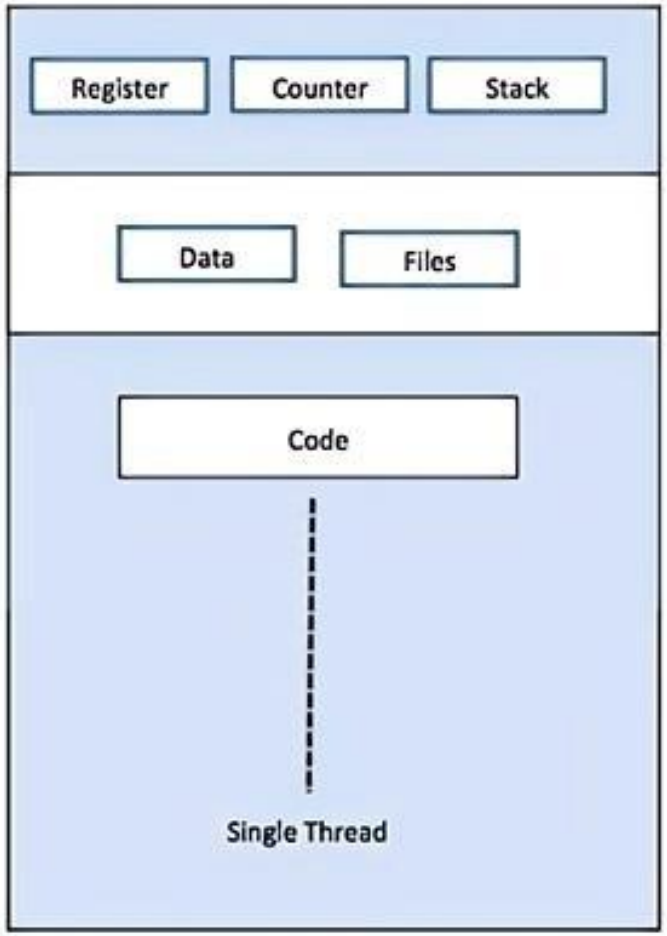
Combined ULT/KLT Model

Single-threaded and Multi-threaded Processes

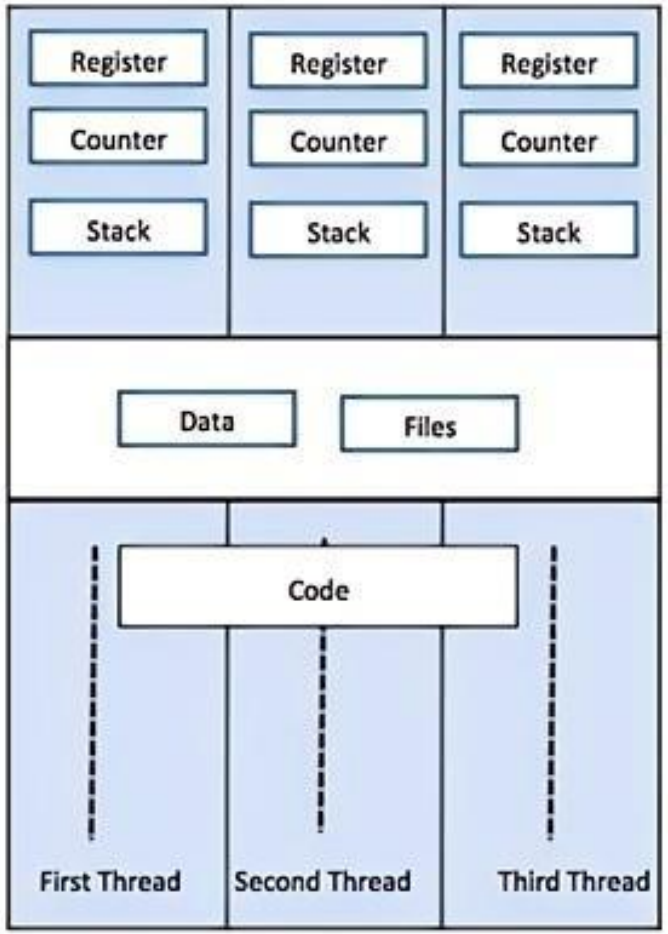
- Single-threaded processes execute instructions sequentially, processing one command at a time in a linear fashion.
- In contrast, multi-threaded processes allow multiple parts of a program to execute concurrently, creating lightweight execution units called threads within the same process space.



The following figure shows the working of a single-threaded and a multithreaded process.



Single Process P with single thread



Single Process P with three threads

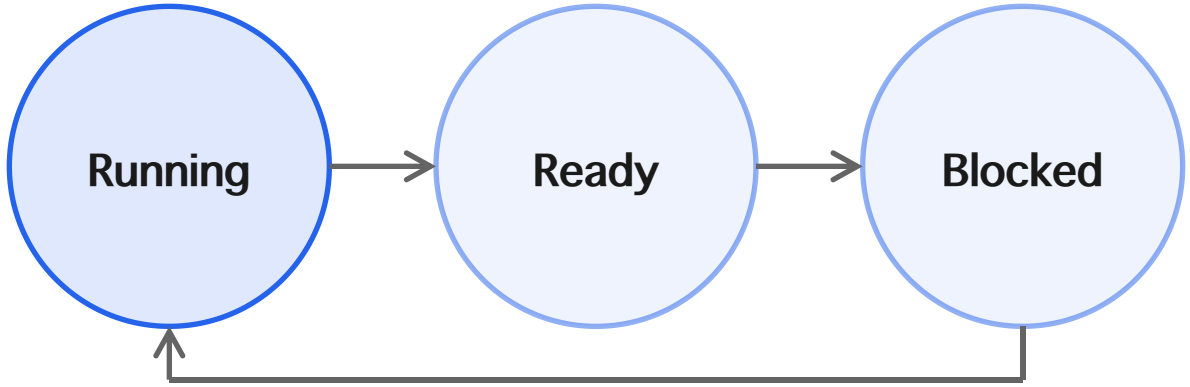
Comparison

Aspect	Single-threaded	Multi-threaded
Execution	Sequential, one task at a time	Concurrent, multiple tasks
Resource Usage	Lower memory overhead	Shared resources, efficient utilization
Responsiveness	Blocks on I/O operations	Remains responsive during blocking
Complexity	Simple to develop and debug	Complex synchronization required
Performance	Limited by sequential execution	Better performance on multicore systems

Multi-threading enables concurrent execution within processes, offering improved performance and responsiveness compared to single-threaded approaches. While implementation complexity increases, the benefits of resource sharing, scalability, and parallelism make multi-threading essential for modern applications, especially on multiprocessor systems.

Thread Execution States

Three Key States



State Transition Operations

- **Spawn** — create another thread
- **Block** — wait for an event
- **Unblock** — event occurred, move to ready
- **Finish** — deallocate register context & stack

Key Question: Will blocking one thread block other threads, or all threads? And what about the *Suspended* state — does it apply to individual threads?

Unlike processes, threads typically do **not** have a separate Suspended state. Suspending a process suspends **all** its threads because they share the same address space. We explore this on the next slide.

Thread Suspend State?

Suspending a process involves suspending **all threads** of that process.

Why?

All threads within a process share the **same address space** and other resources. When the OS swaps out a process (suspends it), the entire address space is moved to disk — there is no way to suspend only one thread while others continue running in the same memory space.

Thus: suspend is a process-level operation, not a thread-level operation.

Thread Synchronization

It is necessary to synchronize the activities of the various threads within a process.

- All threads of a process share the **same address space** and other resources (memory, files, I/O).
- Any alteration of a resource by one thread **affects all other threads** in the same process.
- Without synchronization, race conditions and data corruption can occur when threads access shared data concurrently.

*Detailed synchronization mechanisms (mutexes, semaphores, monitors) are covered in **Chapter 5**.*

Thread — Summary

- Has an **execution state** — Running, Ready, Blocked
- Has its own **execution stack**
- Thread context is **saved and restored** during thread switching
- Has **access to memory and resources** of its parent process
- Has **per-thread static storage** for local variables
- Thread switching is **faster** than process switching (no address space change)

Process — Summary

Virtual Address Space holds the Process Image:

- **Program** and **Data** sections
- **One stack per thread**
- **PCB** (Process Control Block) and **one TCB per thread**

- **Protected access** to memory, files, and I/O resources
- A process contains **one or more threads**
- Process is the unit of **resource allocation**; thread is the unit of **execution**

Benefits of Threads

Faster Creation

Less time to create or terminate a new thread compared to a process. No need to allocate a full address space.

Faster Switching

Less time to switch between two threads within the same process. No TLB flush or address space switch.

Easier Communication

Communication among threads of the same process is easier — shared memory, no IPC overhead.

Observation: Compare the difference in responsiveness when opening multiple windows in Internet Explorer (single-process) vs. a modern multi-threaded browser.

Threads enable **responsive applications**: while one thread handles I/O or a long computation, others can keep the UI responsive.

Symmetric Multiprocessors (SMP)

A stand-alone computer system with the following characteristics:

- Two or more **similar processors** of comparable capability
- Processors share the **same main memory** and are interconnected by a bus or internal connection scheme
- Processors share **access to I/O devices**
- All processors can perform the **same functions** (hence "symmetric")
- Controlled by an **integrated OS** that provides interaction at job, task, file, and data element levels

Multiprocessing = more than one processor in a single machine.

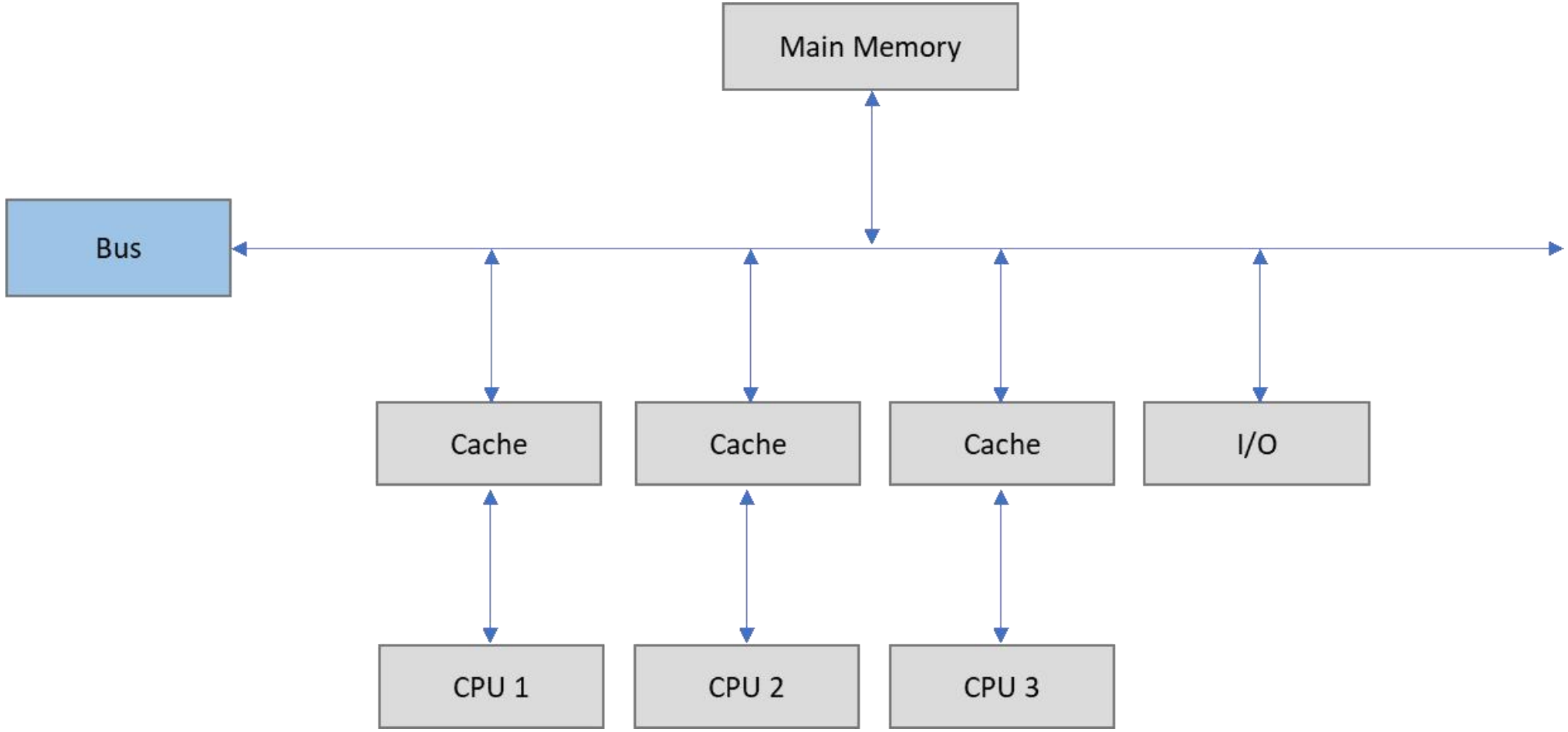
Multicore Computer

Also known as a **chip multiprocessor (CMP)**

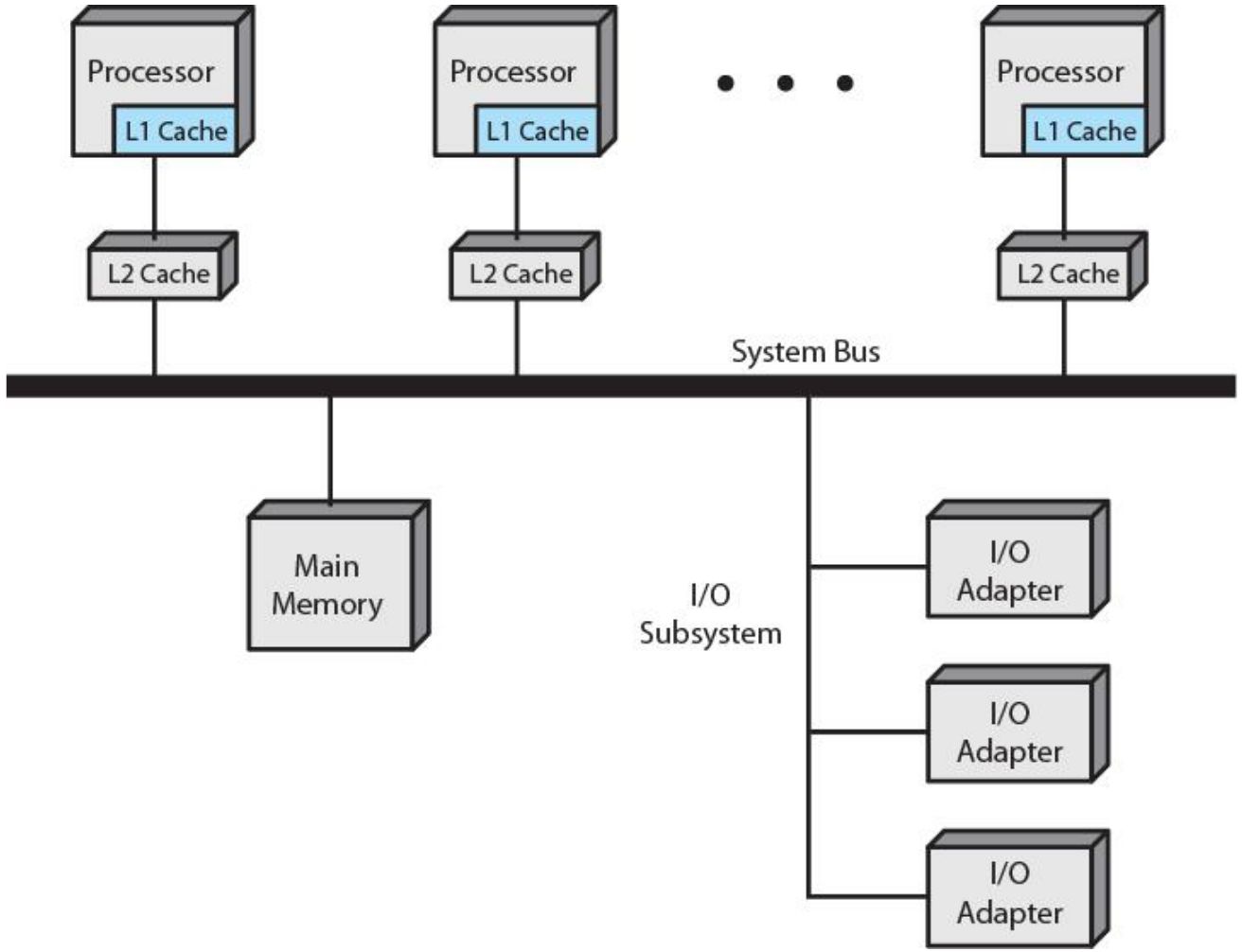
- Combines **two or more processors (cores)** on a single piece of silicon (die)
- Each core contains **all components of an independent processor** — ALU, control unit, registers, L1 cache
- Multicore chips also include **shared L2 cache**, and in some cases **L3 cache**
- Cores communicate via on-chip interconnect — much faster than off-chip SMP bus
- Lower power consumption and higher density than multi-chip SMP

Sharing Memory Space and I/O Across CPUs with SMP

In SMP, all processors or CPU cores are considered equal and share the same system resources like the operating system, the address space of the main memory, and the input/output (I/O) or peripheral devices.



SMP Organization



SMP Architecture: Multiple CPUs sharing main memory and I/O via a common bus